IT Support Cost Optimization

Python script and its output

This document has two parts: the full Python source code (Part 1) and the output produced when the script is run (Part 2). The script uses generated sample data (20,000 tickets, fixed random seed 42), so the results are for demonstration and are not findings about a real company.

Run with: `python it_support_cost_optimization.py` (requires pandas, numpy, scikit-learn)

Part 1: Source code

```
"""
IT Support Cost Optimization
A small, readable analysis script for the assignment.

Install dependencies:
    pip install pandas numpy scikit-learn

Note:
- The assignment image does not include a real ticket dataset, so this script
  creates sample data to demonstrate the analysis.
- Replace make_sample_data() with pd.read_csv("your_tickets.csv") when real
  data is available.
"""

import time
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report


RANDOM_SEED = 42
TICKET_COUNT = 20_000


def make_sample_data(rows=TICKET_COUNT):
    """Create realistic-looking example tickets for a demo."""
    rng = np.random.default_rng(RANDOM_SEED)

    categories = ["Password/Login", "Network", "Software", "Hardware", "Access"]
    priorities = ["Low", "Medium", "High", "Critical"]
    clients = ["Small", "Medium", "Enterprise"]

    category = rng.choice(categories, rows, p=[0.28, 0.20, 0.25, 0.15, 0.12])
    priority = rng.choice(priorities, rows, p=[0.35, 0.40, 0.20, 0.05])
    client_type = rng.choice(clients, rows, p=[0.45, 0.35, 0.20])


    base_hours = {
        "Password/Login": 0.35,
        "Network": 1.8,
        "Software": 1.3,
        "Hardware": 2.2,
        "Access": 0.8,
    }
    priority_factor = {"Low": 0.8, "Medium": 1.0, "High": 1.5, "Critical": 2.0}
    resolution_hours = np.array([
        base_hours[c] * priority_factor[p] for c, p in zip(category, priority)
    ]) * rng.lognormal(mean=0.0, sigma=0.55, size=rows)
    resolution_hours = np.clip(resolution_hours, 0.1, 24)

    hourly_rate = rng.choice([25, 35, 45, 60, 80], rows,
                             p=[0.10, 0.25, 0.30, 0.25, 0.10])
    escalation_count = rng.poisson(
        lam=np.where(priority == "Critical", 1.2,
                     np.where(priority == "High", 0.7, 0.25))
    )
    escalation_count = np.clip(escalation_count, 0, 5)
```

```
    csat = np.clip(
        4.7 - resolution_hours * 0.22 - escalation_count * 0.28
        + rng.normal(0, 0.45, rows),
        1, 5
    ).round(1)

    tickets = pd.DataFrame({
        "issue_category": category,
        "priority": priority,
        "resolution_hours": resolution_hours.round(2),
        "engineer_hourly_cost": hourly_rate,
        "client_type": client_type,
        "escalation_count": escalation_count,
        "csat_score": csat,
    })


    for column in ["issue_category", "priority", "resolution_hours",
                   "engineer_hourly_cost", "client_type", "escalation_count",
                   "csat_score"]:
        missing_rows = rng.choice(rows, size=int(rows * 0.08), replace=False)
        tickets.loc[missing_rows, column] = np.nan

    return tickets


def clean_data(tickets):
    """Fill missing values and calculate estimated ticket cost."""
    data = tickets.copy()

    categorical_columns = ["issue_category", "priority", "client_type"]
    numeric_columns = [
        "resolution_hours", "engineer_hourly_cost",
        "escalation_count", "csat_score"
    ]

    for column in categorical_columns:
        data[column] = data[column].fillna(data[column].mode().iloc[0])

    for column in numeric_columns:
        data[column] = pd.to_numeric(data[column], errors="coerce")
        data[column] = data[column].fillna(data[column].median())


    data["ticket_cost"] = (
        data["resolution_hours"] * data["engineer_hourly_cost"]
    )


    data["possible_unnecessary_escalation"] = (
        data["escalation_count"] >= 2
    ).astype(int)

    return data


def print_summary(data):
    print("\n--- Overall results ---")
    total_cost = data["ticket_cost"].sum()
    print(f"Tickets analyzed: {len(data):,}")
    print(f"Estimated support labour cost: ${total_cost:,.2f}")
    print(f"Average cost per ticket: ${data['ticket_cost'].mean():,.2f}")
    print(f"Average resolution time: {data['resolution_hours'].mean():.2f} hours")
    print(f"Average CSAT score: {data['csat_score'].mean():.2f} / 5")
    print(
```

```
        "Tickets with 2 or more escalations: "
        f"{data['possible_unnecessary_escalation'].mean() * 100:.1f}%"
    )

    print("\n--- Cost by issue category ---")
    category_report = (
        data.groupby("issue_category")
        .agg(
            ticket_count=("ticket_cost", "size"),
            total_cost=("ticket_cost", "sum"),
            avg_cost=("ticket_cost", "mean"),
            avg_resolution_hours=("resolution_hours", "mean"),
            avg_csat=("csat_score", "mean"),
        )
        .sort_values("total_cost", ascending=False)
        .round(2)
    )
    print(category_report.to_string())

    print("\n--- Cost by client type ---")
    client_report = (
        data.groupby("client_type")
        .agg(
            ticket_count=("ticket_cost", "size"),
            total_cost=("ticket_cost", "sum"),
            avg_cost=("ticket_cost", "mean"),
            avg_csat=("csat_score", "mean"),
        )
        .sort_values("total_cost", ascending=False)
        .round(2)
    )
    print(client_report.to_string())

    print("\n--- Cost by priority ---")
    priority_report = (
        data.groupby("priority")
        .agg(
            ticket_count=("ticket_cost", "size"),
            total_cost=("ticket_cost", "sum"),
            avg_resolution_hours=("resolution_hours", "mean"),
            avg_csat=("csat_score", "mean"),
        )
        .sort_values("total_cost", ascending=False)
        .round(2)
    )
    print(priority_report.to_string())


    print("\n--- Savings target ---")
    print(f"20% reduction target: ${total_cost * 0.20:,.2f}")
    print(f"Target cost after reduction: ${total_cost * 0.80:,.2f}")


def train_escalation_model(data):
    """Predict whether a ticket may need repeated escalation."""
    features = [
        "issue_category", "priority", "resolution_hours",
        "engineer_hourly_cost", "client_type"
    ]
    target = "possible_unnecessary_escalation"

    X = data[features]
    y = data[target]

    if y.nunique() < 2:
        print("\nNot enough variation in escalation labels to train a model.")
        return None
```

```

    categorical_features = ["issue_category", "priority", "client_type"]
    numeric_features = ["resolution_hours", "engineer_hourly_cost"]

    preprocessing = ColumnTransformer(
        transformers=[
            ("categories", Pipeline(steps=[
                ("fill", SimpleImputer(strategy="most_frequent")),
                ("encode", OneHotEncoder(handle_unknown="ignore")),
            ]), categorical_features),
            ("numbers", Pipeline(steps=[
                ("fill", SimpleImputer(strategy="median")),
            ]), numeric_features),
        ]
    )

    model = Pipeline(steps=[
        ("preprocessing", preprocessing),
        ("classifier", RandomForestClassifier(
            n_estimators=100,
            max_depth=10,
            class_weight="balanced",
            random_state=RANDOM_SEED,
            n_jobs=-1,
        )),
    ])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_SEED, stratify=y
    )

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("\n--- Escalation model evaluation ---")
    print(classification_report(y_test, predictions, zero_division=0))


    sample_ticket = X_test.iloc[[0]]
    start = time.perf_counter()
    model.predict(sample_ticket)
    prediction_seconds = time.perf_counter() - start
    print(f"Single-ticket prediction time: {prediction_seconds:.4f} seconds")

    return model


def print_recommendations(data):
    """Turn the summary into a few practical next steps."""
    print("\n--- Recommendations to investigate ---")

    category_cost = data.groupby("issue_category")["ticket_cost"].sum()
    highest_cost_category = category_cost.idxmax()
    print(
        f"1. Start with '{highest_cost_category}', the highest-cost issue "
        "category in this sample. Check whether self-service guides or "
        "automation can resolve common tickets."
    )

    category_time = data.groupby("issue_category")["resolution_hours"].mean()
    slowest_category = category_time.idxmax()
    print(
        f"2. Review the workflow for '{slowest_category}', which has the "
        "longest average resolution time in this sample."
    )

    repeated_escalations = data[data["escalation_count"] >= 2]
```

```
    print(
        f"3. Audit the {len(repeated_escalations):,} tickets with two or more "
        "escalations. Confirm whether they were avoidable before changing "
        "the escalation policy."
    )

    low_csat = data[data["csat_score"] < 4.2]
    print(
        f"4. Review low-satisfaction cases ({len(low_csat):,} in this sample). "
        "Use actual customer survey data before reporting against the 4.2 goal."
    )

    print(
        "5. Pilot one change at a time and compare cost, resolution time, "
        "escalation rate, and CSAT before and after the change."
    )


def main():
    raw_tickets = make_sample_data()
    tickets = clean_data(raw_tickets)

    print("IT Support Cost Optimization - Sample Analysis")
    print_summary(tickets)
    train_escalation_model(tickets)
    print_recommendations(tickets)

    print(
        "\nImportant: these figures use generated sample data. "
        "They are not findings about the real company."
    )
    print(
        "A Python script alone does not prove the system can process "
        "50,000 events per second. For production streaming, use a streaming "
        "platform such as Kafka with a stream processor, benchmark it, and "
        "monitor end-to-end latency against the 5-second requirement."
    )


if __name__ == "__main__":
    main()
```

Part 2: Output

```
IT Support Cost Optimization - Sample Analysis

--- Overall results ---
Tickets analyzed: 20,000
Estimated support labour cost: $1,412,501.70
Average cost per ticket: $70.63
Average resolution time: 1.49 hours
Average CSAT score: 4.24 / 5
Tickets with 2 or more escalations: 6.3%

--- Cost by issue category ---
                ticket_count  total_cost  avg_cost  avg_resolution_hours  avg_csat
issue_category
Network                 3580   371463.30    103.76                  2.18      4.10
Software                4671   355316.10     76.07                  1.61      4.23
Hardware                2736   347145.25    126.88                  2.67      3.99
Password/Login          6790   234141.55     34.48                  0.73      4.40
Access                  2223   104435.50     46.98                  1.00      4.35

--- Cost by client type ---
             ticket_count  total_cost  avg_cost  avg_csat
client_type
Small                9890   690158.65     69.78      4.25
Medium               6458   462507.85     71.62      4.23
Enterprise           3652   259835.20     71.15      4.24

--- Cost by priority ---
          ticket_count  total_cost  avg_resolution_hours  avg_csat
priority
Medium            8948   588396.25                  1.39      4.29
High              3779   364250.85                  2.04      4.05
Low               6354   336633.20                  1.12      4.35
Critical           919   123221.40                  2.82      3.78

--- Savings target ---
20% reduction target: $282,500.34
Target cost after reduction: $1,130,001.36

--- Escalation model evaluation ---
              precision    recall  f1-score   support

           0       0.96      0.81      0.88      3749
           1       0.16      0.56      0.25       251

    accuracy                           0.79      4000
   macro avg       0.56      0.68      0.57      4000
weighted avg       0.91      0.79      0.84      4000

Single-ticket prediction time: 0.0422 seconds

--- Recommendations to investigate ---
1. Start with 'Network', the highest-cost issue category in this sample. Check whether self-service guides or
   automation can resolve common tickets.
2. Review the workflow for 'Hardware', which has the longest average resolution time in this sample.
3. Audit the 1,257 tickets with two or more escalations. Confirm whether they were avoidable before changing the
   escalation policy.
4. Review low-satisfaction cases (7,363 in this sample). Use actual customer survey data before reporting against
   the 4.2 goal.
5. Pilot one change at a time and compare cost, resolution time, escalation rate, and CSAT before and after the
   change.

Important: these figures use generated sample data. They are not findings about the real company.
A Python script alone does not prove the system can process 50,000 events per second. For production streaming, use
   a streaming platform such as Kafka with a stream processor, benchmark it, and monitor end-to-end latency against
   the 5-second requirement.
```